



Karyvio

LEARN | BUILD | GROW | GET HIRED

Better
Developers
Brighter
Futures 😊

Complete

MERN Stack

Handwritten Study Guide

From Basics to Production



MongoDB

ex

Express.js



React.js



Node.js

- ✓ Concepts
- ✓ Code Examples
- ✓ Real-World Projects
- ✓ Interview Questions
- ✓ Cheat Sheets
- ✓ Deployment Guide

"Learn Today
Build Tomorrow"
😊

Code
Practice
Get Hired



Karyvio.com

Your Learning Partner in Tech Journey

Small
Steps
Big Growth

MERN Stack

Complete Notes

Beginner to Advanced

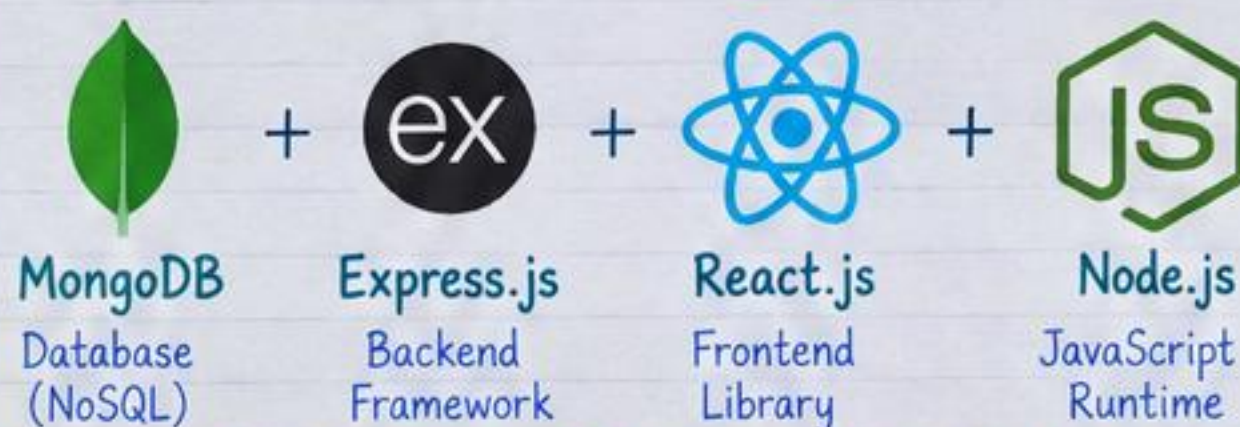
A Complete Guide to Build Modern Web Applications

Same
Technology
Different Dreams
Keep Learning
😊

1 What is MERN Stack ?

- Ans →
- MERN stack is a combination of four powerful technologies used to build modern, full-stack web applications.
 - It stands for MongoDB, Express.js, React.js and Node.js.
 - It allows you to build both frontend and backend using JavaScript.
 - MERN is popular because it is fast, flexible, scalable and has a large community.

MERN Technologies



2 Why Use MERN Stack ?

- Ans →
- Uses a single language (JavaScript) for both frontend and backend.
 - Large community support.
 - Open source and free to use.
 - Fast development with modern tools.
 - Highly scalable and flexible.
 - Great for real-world applications (startups, companies, products).

Key Features :-

- Full-stack with JavaScript
- Component based UI (React)
- RESTful API with Express)
- NoSQL database (MongoDB)
- Fast and developer friendly
- Large ecosystem (npm)
- Highly scalable
- Suitable for modern web apps

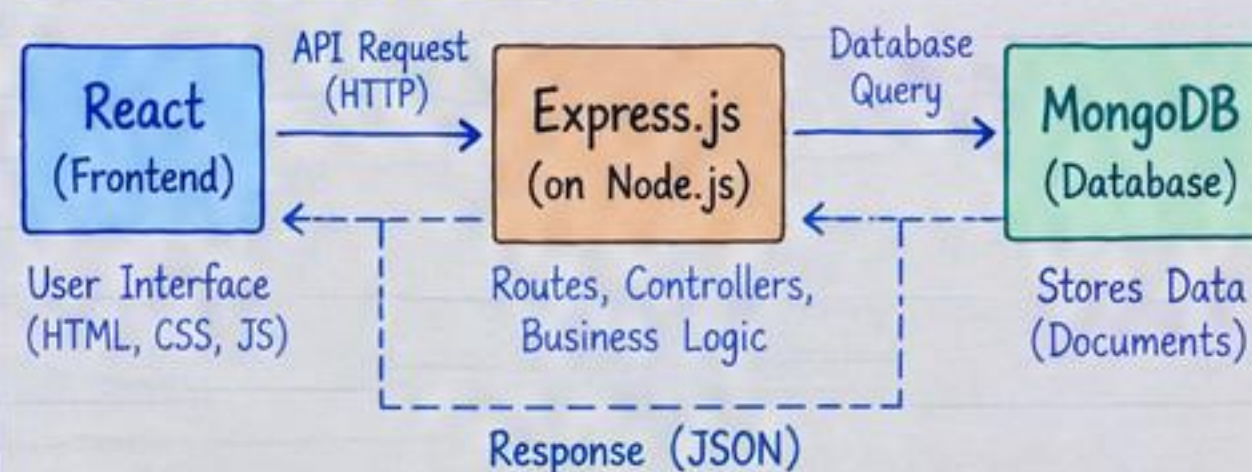
Used For :-

- Web applications
- E-commerce platforms
- Social media apps
- Real-time applications
- Admin dashboards
- Job portals
- Learning platforms
- SaaS products
- and many more ...

3 How MERN Works Together ?

- Ans →
- React handles the user interface (frontend).
 - It sends API requests to the Express server.
 - Express (running on Node.js) handles the requests, business logic and communicates with MongoDB.
 - MongoDB stores the data in a flexible format (documents).
 - The server sends the response back to React, which updates the UI.

MERN Architecture (Flow)



4 Real World Examples Using MERN

- Ans →
- Facebook (uses React)
 - Instagram (uses React)
 - Netflix (uses React)
 - Uber (uses Node.js)
 - LinkedIn (uses React)
 - Many startups and modern companies use MERN for their products.

Benefits for Developers :-

- Learn once, use everywhere (JavaScript)
- Many learning resources available
- Active community support
- Easy to find jobs and opportunities
- Great for building your portfolio
- Can build real-world projects
- Frequently used in industry

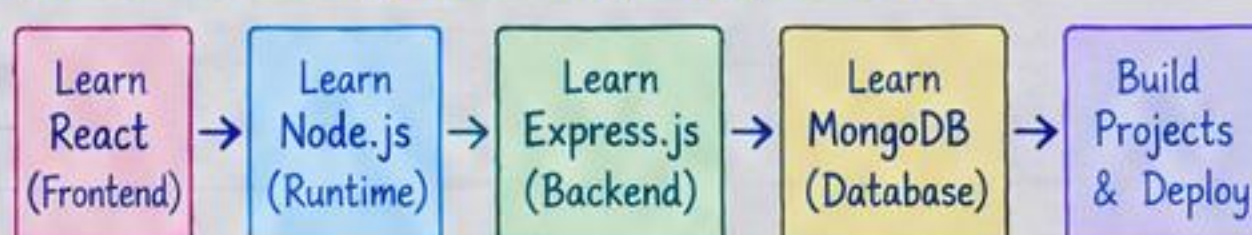
Important Note :-

- MERN is not a full framework, it's a stack of technologies.
- You can replace any technology (e.g. Next.js instead of React) based on project needs.
- MERN is a great choice for beginners and professionals both.

5 What You Will Learn in These Notes ?

- Ans →
- Frontend development with React
 - Backend development with Node.js and Express
 - Database management with MongoDB
 - Authentication, authorization and security
 - Building real-world MERN projects
 - Deployment and industry best practices
 - Interview questions and cheat sheets

MERN Learning Roadmap (High Level)



"Small Steps Everyday Make You a Better Developer!" 😊

React Setup & Project Structure

Set up your first React project and understand the folder structure

Good Setup Leads to Great Projects !



1 Requirements Before You Start ?

- Ans →
- A computer (Windows / macOS / Linux)
 - Node.js (v18 or higher)
 - npm (comes with Node.js)
 - A code editor (VS Code recommended)
 - Basic knowledge of HTML, CSS and JavaScript

Check Versions :-

```
node -v // e.g. v20.11.0
npm -v // e.g. 10.2.4
npx -v // e.g. 10.2.4
```

Make sure Node.js is installed properly.



2 Create a New React Project (Using Vite)

- Ans →
- Vite is a modern and fast build tool for React.
 - Open your terminal and run the following command:

```
npm create vite@latest my-react-app -- --template react
```

- Now go to the project folder: `cd my-react-app`
- Install the dependencies: `npm install`
- Start the development server: `npm run dev`

Explanation :-

- `npm create vite@latest` → creates a new React project using Vite.
- `--template react` → uses React template.
- `npm install` → installs all required dependencies.
- `npm run dev` → starts the development server (usually on `http://localhost:5173`).

3 Project Folder Structure

- Ans →
- A clean and organized folder structure helps in better development and maintenance.
 - Vite provides a simple and modern structure.
 - You can customize it based on your project need.

Folder Structure (Vite + React)

my-react-app/	
node_modules/	# Project dependencies
public/	# Static files (images, favicon, etc.)
src/	# Source code
assets/	# Images, fonts, etc.
components/	# Reusable components
App.jsx	# Main component
main.jsx	# Entry point
index.html	# HTML template
package.json	# Project metadata and scripts
vite.config.js	# Vite configuration
README.md	# Project documentation

4 Important Files Explained

Ans →

File / Folder	Purpose
index.html	Main HTML file (root template)
src/main.jsx	Entry point of the React app
src/App.jsx	Main component (you can modify this)
src/components/	Reusable components
src/assets/	Static assets like images
public/	Files that are directly served
vite.config.js	Vite configuration file
package.json	Project details and dependencies

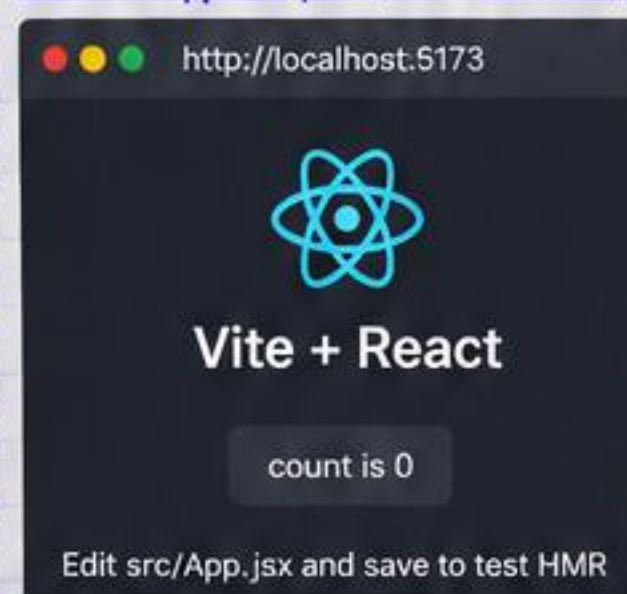
Useful npm Scripts

```
npm run dev # start dev server
npm run build # build for production
npm run preview # preview build
npm install # install dependencies
```

5 Run Your First React App

- Ans →
- After running `npm run dev`, open the URL (usually `http://localhost:5173`) in your browser.
 - You will see a default React + Vite page.
 - You can now start modifying the code in `src/App.jsx`.
 - Save the file and the browser will automatically reload (Hot Module Replacement).

Default App Output (Vite + React)



Pro Tips :-

- Use VS Code with useful extensions (ES7+ React, Prettier, ESLint)
- Keep components small and reusable.
- Follow a consistent folder structure.
- Use meaningful file and folder names.
- You can use Tailwind CSS for styling (we'll cover it later).



React Fundamentals

Learn the core concepts of React to build UI components

Strong Fundamentals Build Great Projects !
😊

1 What is JSX ?

- Ans →
- JSX (JavaScript XML) is a syntax extension for JavaScript.
 - It allows us to write HTML-like code inside JavaScript.
 - JSX makes it easier to create and visualize UI components.
 - It gets transformed into regular JavaScript (using Babel) before running in the browser.

Example :-

```
// JSX Example
const element = (
  <h1 className="title">
    Hello React !
  </h1>
);
```

Looks like HTML but it's JavaScript (JSX)

2 React Components

- Ans →
- Components are the building blocks of a React app.
 - A component is a reusable piece of UI.
 - There are two types of components:
 - Function Components (modern, preferred).
 - Class Components (old, rarely used)
 - Components help in building modular and maintainable code.

Function Component Example :-

```
function Welcome() {
  return <h2>Welcome to Karyvio!</h2>;
}

export default Welcome;
```

Simple Function Component

3 Props (Properties)

- Ans →
- Props allow us to pass data from a parent component to a child component.
 - They are read-only (cannot be modified by the child).
 - Props make components reusable and dynamic.

Example :-

```
function UserCard({ name, age }) {
  return (
    <div>
      <h3>{name}</h3>
      <p>Age: {age}</p>
    </div>
  );
}
```

Using the Component :-

```
<UserCard
  name="Sovit"
  age={21} />
```

Passing data via props

4 State

- Ans →
- State is used to store and manage data inside a component.
 - When state changes, React re-renders the component automatically.
 - We use the useState hook to create state in function components.

Example :-

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

Click button to update state and re-render the component

5 Events

- Ans →
- Events allow us to handle user interactions like clicks, input changes, form submission etc.
 - We use camelCase event names (e.g. onClick, onChange) in React.

Example :-

```
function Button() {
  const handleClick = () => {
    alert("Button Clicked!");
  };

  return <button onClick={handleClick}>
    Click Me
  </button>;
}
```

Handle click event in React

6 Conditional Rendering

- Ans →
- Used to render different UI based on conditions.
 - We can use if-else, ternary operator, or logical && operator.

Example (Ternary) :-

```
function Message({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? (
        <p>Welcome Back!</p>
      ) : (
        <p>Please Login</p>
      )}
    </div>
  );
}
```

Render different UI based on condition

Common Event List :-

- onClick
- onChange
- onSubmit
- onFocus
- onBlur
- onKeyDown
- onKeyUp
- onMouseEnter
- onMouseLeave
- and many more ...

React Performance & Best Practices

Build faster, cleaner and scalable React applications

Better
Performance
Better Users
Happier You !
😊

1 Why Performance Matters ?

- Ans →
- Faster loading and smoother interactions.
 - Better user experience and higher retention.
 - Improves SEO (Core Web Vitals).
 - Reduces unnecessary re-renders and memory usage.
 - Important for large and data-heavy applications.

Key Metrics (Web Vitals) :-

- LCP (Largest Contentful Paint)
- loading performance
- FID (First Input Delay)
- interactivity
- CLS (Cumulative Layout Shift)
- visual stability

Tools to Measure :-

- React DevTools (Profiler)
- Lighthouse
- Web Vitals Extension
- Chrome Performance Tab
- Why Did You Render
- and more ...

2 Avoid Unnecessary Re-renders

- Ans →
- Re-renders can be expensive, especially in large apps.
 - Use `React.memo`, `useMemo` and `useCallback` to optimize.
 - Keep state as local as possible.
 - Avoid inline functions and objects in JSX.

Example :- `React.memo`

```
import React from "react";

const UserCard = React.memo(({ name }) => {
  console.log("Rendered:", name);
  return <div>{name}</div>;
});

export default UserCard;
```

Component
will re-render
only if props
change

3 Use `useMemo` for Expensive Calculations

- Ans →
- `useMemo` caches the result of a calculation.
 - Useful for expensive computations.
 - Re-calculates only when dependencies change.

Example :- `useMemo`

```
import { useMemo, useState } from "react";

function ExpensiveComponent({ items }) {
  const sortedItems = useMemo(() => {
    console.log("Sorting items...");
    return [...items].sort((a, b) =>
      a.name.localeCompare(b.name));
  }, [items]);

  return <ul>{sortedItems.map(item => <li key={item.id}>{item.name}</li>)}</ul>;
}
```

Sorting happens
only when
items change

4 Use `useCallback` for Stable Functions

- Ans →
- `useCallback` memoizes a function reference.
 - Useful when passing functions to child components.
 - Prevents unnecessary re-renders in child components.

Example :- `useCallback`

```
import { useCallback, useState } from "react";

const Button = ({ onClick }) => {
  console.log("Button rendered");
  return <button onClick={onClick}>Click Me</button>;
};

function App() {
  const [count, setCount] = useState(0);
  const handleClick = useCallback(() => {
    setCount(c => c + 1);
  }, []);

  return <Button onClick={handleClick} />;
}
```

Same function
reference
across renders

5 Code Splitting & Lazy Loading

- Ans →
- Load components only when needed.
 - Reduces initial bundle size.
 - Use `React.lazy` and `Suspense`.

Example :- Lazy Loading

```
import { lazy, Suspense } from "react";

const About = lazy(() => import("../pages/About"));

function App() {
  return (
    <div>
      <h1>My App</h1>
      <Suspense fallback={<p>Loading...</p>}>
        <About />
      </Suspense>
    </div>
  );
}
```

Component
is loaded
only when
needed

6 Optimize Images and Assets

- Ans →
- Use modern formats (WebP, AVIF).
 - Compress images.
 - Use lazy loading for images.
 - Use CDN for static assets.

Example :- Lazy Image

```

```

Lazy loading
improves
page load time

Tips :-

- Use WebP format
- Resize images
- Use next-gen formats
- Use a CDN
- Lazy load below the fold
- Remove unused assets

7 Other Best Practices

- Ans →
- Keep components small and focused.
 - Use proper folder structure.
 - Avoid deep prop drilling (use Context or state management).
 - Remove unused code and dependencies.
 - Use TypeScript for better maintainability.
 - Follow consistent naming and code style.
 - Use ESLint and Prettier.
 - Regularly update dependencies.

Do's and Don'ts :-

✓ Do's

- Use memorization wisely
- Keep state local
- Use lazy loading
- Optimize assets
- Follow modern React patterns
- Monitor performance regularly

✗ Don'ts

- Don't overuse `useMemo`/`useCallback`
- Don't put everything in global state
- Don't ignore re-render warnings
- Don't load large assets unnecessarily
- Don't create inline functions in JSX
- Don't sacrifice readability for optimization



API Error Handling in Express.js

Build reliable and user-friendly APIs with proper error handling

Good Error Handling Makes Your API Stable, Secure and Developer Friendly!

1 Why Error Handling ?

- Ans →
- Prevents server crashes
 - Provides meaningful error messages
 - Improves user experience
 - Helps in debugging and monitoring
 - Makes APIs more secure and reliable

2 Types of Errors

- Ans →
- Synchronous errors (try...catch)
 - Asynchronous errors (promises, async/await)
 - Validation errors (invalid input)
 - Not found errors (resource not found)
 - Authentication/Authorization errors
 - Database errors (e.g. MongoDB, PostgreSQL)
 - Custom application errors

3 HTTP Status Codes

Ans →

Code	Meaning
200	OK (Success)
400	Bad Request (Invalid input)
401	Unauthorized (Invalid token)
403	Forbidden (No permission)
404	Not Found (Resource not found)
409	Conflict (Already exists)
500	Internal Server Error (Server issue)

4 Basic Try-Catch (Synchronous)

Ans →

```
app.get('/sync-error', (req, res) => {
  try {
    const x = 10 / 0;
    res.send('Result: ' + x);
  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Something went wrong',
      error: error.message
    });
  }
});
```

5 Handling Async Errors

Ans →

```
app.get('/async-error', async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) {
      return res.status(404).json({
        success: false,
        message: 'User not found'
      });
    }
    res.json({ success: true, data: user });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Server error',
      error: error.message
    });
  }
});
```

6 Async Handler (Cleaner)

Ans → We can create a wrapper to catch async errors automatically.

```
// utils/asyncHandler.js

const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

// Usage
const asyncHandler = require('./utils/asyncHandler');

app.get('/users/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id);
  if (!user) return res.status(404).json({
    message: 'User not found'
  });
  res.json(user);
})));
```

7 Global Error Handling Middleware

Ans →

```
// Place this after all routes
app.use((err, req, res, next) => {
  console.error(err.stack);

  const status = err.statusCode || 500;
  const message = err.message || 'Internal Server Error';

  res.status(status).json({
    success: false,
    message,
    stack: process.env.NODE_ENV === 'development'
      ? err.stack
      : undefined
  });
});
```

8 Custom Error Class

Ans → Create a reusable error class for consistent errors.

```
// utils/AppError.js

class AppError extends Error {
  constructor(message, statusCode = 500, isOperational = true) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = isOperational;
    Error.captureStackTrace(this, this.constructor);
  }
}

module.exports = AppError;

// Usage in route
const AppError = require('./utils/AppError');
if (!user) {
  throw new AppError('User not found', 404);
}
```

9 Handling Validation Errors

Ans → Using express-validator (or any validation library).

```
const { body, validationResult } = require('express-validator');

app.post('/users',
  body('name').notEmpty().withMessage('Name is required'),
  (req, res, next) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({
        success: false,
        message: 'Validation failed',
        errors: errors.array()
      });
    }
    // create user...
  })
```

10 Best Practices

- Ans →
- Use meaningful error messages.
 - Use correct HTTP status codes.
 - Create a global error handler.
 - Use a custom error class.
 - Handle both sync and async errors.
 - Do not expose sensitive error details in production.
 - Log errors (use tools like Winston, Morgan, etc.).
 - Validate input data.
 - Keep error response format consistent.
 - Test error scenarios.

Example Error Response

```
{
  "success": false,
  "message": "User not found",
  "error": "User with this ID does not exist"
}
```

Next Topics :-

- Logging with Winston / Morgan
- Input Validation (Joi / express-validator)
- Authentication & Authorization Errors
- Database Error Handling (MongoDB / PostgreSQL)
- API Response Formatting
- Monitoring (Sentry, Logs, APM)
- Building a Complete Error Handling Module

MongoDB Advanced Queries

Powerful query techniques to handle real-world data efficiently

Query Smarter
Handle Large Data
Build Scalable Apps
with MongoDB

1 What are Advanced Queries?

Ans → Advanced queries go beyond basic CRUD to perform complex data retrieval, filtering, transformation and analysis.

- Used for real-world applications with large and unstructured data.
- Helps in building search, analytics, reporting and scalable features.

2 Query Operators Overview

Ans →

Category	Operators	Purpose
Comparison	\$eq, \$ne, \$gt, \$gte, \$lt, \$lte	Compare values
Logical	\$and, \$or, \$not	Combine conditions
Element	\$in, \$nin	Match in array
Evaluation	\$exists, \$type	Check field existence/type
Array	\$all, \$elemMatch, \$size	Work with arrays
Text	\$text	Full-text search
Geospatial	\$geoNear, \$geoWithin	Location based queries

3 Comparison Operators

Ans →

```
// Find users older than 22
db.users.find({ age: { $gt: 22 } })

// Find users with age between 18 and 25
db.users.find({
  age: { $gte: 18, $lte: 25 }
})
```

Common Comparison Operators:

\$eq → Equal	\$lt → Less than
\$ne → Not equal	\$lte → Less than or equal
\$gt → Greater than	\$in → In an array
\$gte → Greater than or equal	\$nin → Not in an array

4 Logical Operators

Ans →

```
// Users age > 18 AND skill = Node.js
db.users.find({
  $and: [
    { age: { $gt: 18 } },
    { "skills": "Node.js" }
  ]
})

// Users from Odisha OR remote location
db.users.find({
  $or: [
    { state: "Odisha" },
    { workMode: "Remote" }
  ]
})
```

5 Querying Arrays

Ans →

```
// Users with a specific skill (element in array)
db.users.find({ skills: "JavaScript" })

// Users with multiple skills
db.users.find({
  skills: { $all: ["JavaScript", "Node.js"] }
})

// Using $elemMatch for complex array query
db.users.find({
  skills: {
    $elemMatch: { name: "MongoDB", level: "Advanced" }
  }
})

// Find users with exactly 3 skills
db.users.find({ skills: { $size: 3 } })
```

6 Projection (Select Fields)

Ans →

```
// Return only name and email
db.users.find(
  {},
  { name: 1, email: 1, _id: 0 }
)

// Exclude sensitive fields
db.users.find(
  {},
  { password: 0, refreshToken: 0 }
)

// Rename a field using aggregation (see next)
// Projection keeps responses smaller and faster.
```

Tip : • 1 = include field
• 0 = exclude field
(You cannot mix 1 and 0, except _id)

7 Sorting, Limiting & Skipping

Ans →

```
// Get latest 10 users
db.users.find()
.sort({ createdAt: -1 }) // -1 desc, 1 asc
.limit(10)

// Skip first 20 and get next 10
db.users.find()
.sort({ createdAt: -1 })
.skip(20)
.limit(10)
```

8 Text Search

Ans →

```
// Create text index (run once)
db.posts.createIndex({
  title: "text",
  content: "text"
})

// Search posts
db.posts.find({
  $text: { $search: "mongodb tutorial" }
})

// Text search with projection and sort
db.posts.find(
  { $text: { $search: "node mongodb" } },
  { score: { $meta: "textScore" } }
).sort({ score: { $meta: "textScore" } })
```

9 Aggregation Pipeline

Ans →

```
// Count users by skill
db.users.aggregate([
  { $unwind: "$skills" },
  { $group: { _id: "$skills", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])

// Get average age by country
db.users.aggregate([
  { $group: { _id: "$country", avgAge: { $avg: "$age" } } },
  { $sort: { avgAge: -1 } }
])
```

10 Geospatial Queries

Ans →

```
// Find users within 5 km of a location
db.users.find({
  location: {
    $near: {
      $geometry: { type: "Point", coordinates: [85.8245, 20.2961] },
      $maxDistance: 5000
    }
  }
})
```

11 Real-World Use Case

Ans →

Example: Job Search

- ✓ Filter by location, skills, experience range using advanced queries.
- ✓ Use text search for job title/description.
- ✓ Apply sorting, pagination and projection for better performance.
- ✓ Use aggregation for analytics (e.g., top skills, jobs by company, etc.).

12 Best Practices

Ans →

- Use indexes for frequently queried fields.
- Use projection to fetch only required data.
- Avoid deep nesting in queries.
- Use aggregation for complex analytics.
- Validate input to prevent NoSQL injection.
- Keep queries efficient and optimized.
- Use explain() to analyze query performance.
- Follow schema design best practices.
- Test queries with realistic data.



File Uploads & Images

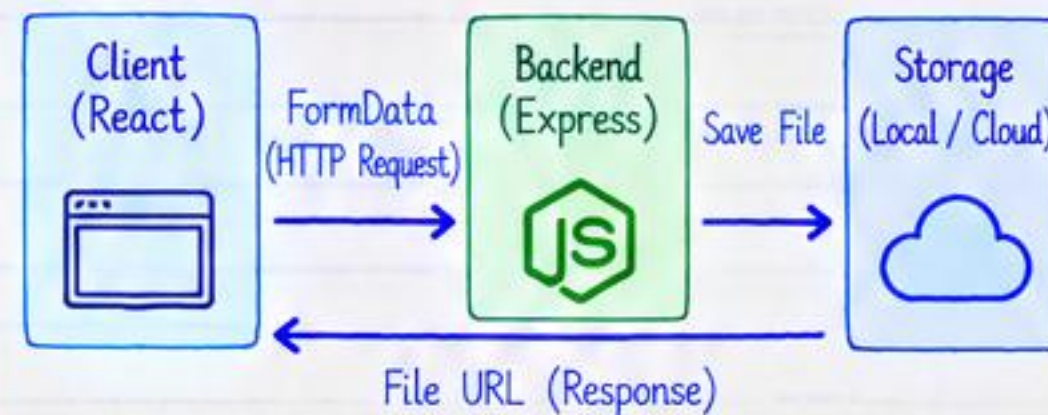
Handle file uploads, store images and manage media in MERN applications

Upload
Process
Store
Serve
Use in App
😊

1 Why File Uploads ?

- Allow users to upload images, documents and other files.
- Common in profiles, blogs, e-commerce, resumes, etc.
- Need proper validation, storage and security.
- Can store files locally or in cloud (Cloudinary, AWS S3, etc.).

2 Upload Flow (Overview)



3 Install Required Packages

Backend (Express)

```
$ npm install multer
$ npm install cloudinary
$ npm install dotenv
```

Frontend (React)

```
$ npm install axios
```

4 Configure Multer (Local Storage)

```
// backend/config/multer.js
const multer = require('multer');
const path = require('path');

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads');
  },
  filename: (req, file, cb) => {
    const uniqueName = Date.now() +
      path.extname(file.originalname);
    cb(null, uniqueName);
  }
});

const upload = multer({
  storage,
  limits: { fileSize: 5 * 1024 * 1024 }
});

module.exports = upload;
```

5 Upload Route (Express)

```
// backend/routes/upload.js
const express = require('express');
const upload = require('../config/multer');
const router = express.Router();

router.post('/upload',
  upload.single('image'), (req, res) => {
    if (!req.file) {
      return res.status(400).json({
        message: 'No file uploaded'
      });
    }

    const filePath = "/uploads/${req.file.filename}";
    res.status(200).json({
      message: 'File uploaded successfully',
      filePath
    });
  });

module.exports = router;
```

6 Upload to Cloudinary (Cloud Storage)

```
// backend/config/cloudinary.js
const cloudinary = require('cloudinary').v2;
require('dotenv').config();

cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD,
  api_key: process.env.CLOUDINARY_KEY,
  api_secret: process.env.CLOUDINARY_SECRET
});

module.exports = cloudinary;

// backend/routes/uploadCloud.js
const router = require('express').Router();
const upload = require('../config/multer');
const cloudinary = require('../config/cloudinary');

router.post('/upload', upload.single('image'),
  async (req, res) => {
    const result = await cloudinary.uploader.upload(
      req.file.path, { folder: 'karyvio' }
    );
    res.json({ url: result.secure_url });
  });

module.exports = router;
```

7 Frontend: Upload Image (React)

```
// src/components/UploadImage.jsx
import { useState } from 'react';
import axios from 'axios';

function UploadImage() {
  const [file, setFile] = useState(null);
  const [url, setUrl] = useState('');

  const handleUpload = async () => {
    const formData = new FormData();
    formData.append('image', file);
    const res = await axios.post(
      'http://localhost:5000/upload', formData,
      { headers: { 'Content-Type': 'multipart/form-data' } }
    );
    setUrl(res.data.filePath);
  };

  return (
    <div>
      <input type="file" onChange={(e) => setFile(e.target.files[0])} />
      <button onClick={handleUpload}>Upload</button>
      {url && <img src={url} alt="Uploaded" width="200" />}
    </div>
  );
}

export default UploadImage;
```

8 Store File Info in MongoDB

```
// models/File.js
const mongoose = require('mongoose');

const fileSchema = new mongoose.Schema({
  filename: String,
  url: String,
  uploadedBy: { type: mongoose.Schema.Types.ObjectId,
    ref: 'User' },
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('File', fileSchema);
```

9 Validation & Security

- ✓ Check file type (only images, PDF, etc.).
- ✓ Set file size limit.
- ✓ Use unique file names.
- ✓ Validate on both frontend and backend.
- ✓ Use cloud storage for production.
- ✓ Prevent direct access to sensitive files.
- ✓ Sanitize file names.

10 Best Practices

- ✓ Use cloud storage (Cloudinary / AWS S3) for scalability.
- ✓ Optimize images (resize/compress).
- ✓ Store only file URL in database.
- ✓ Use environment variables for keys.
- ✓ Restrict file types and size.
- ✓ Add authentication (only logged-in users).
- ✓ Delete unused files.
- ✓ Use CDN for faster delivery.

11 Next Topics

- Image Optimization
- Multiple File Uploads
- Drag & Drop Upload UI
- Progress Bar (Upload)
- Delete / Update Files
- Secure File Access (Signed URLs)
- AWS S3 Integration
- Handling Other File Types (PDF, Docs, etc.)



Same
Code
Bigger
Opportunities
😊

MERN Deployment & Environment Configuration

Take your MERN app from local to the world

Code
Deploy
Scale
Grow
😊

1 Deployment Options

You can deploy MERN apps on many platforms. Choose based on your needs.

- ✓ Frontend → Vercel, Netlify, Cloudflare Pages
- ✓ Backend → Render, Railway, Fly.io, AWS, DigitalOcean
- ✓ Database → MongoDB Atlas (cloud)
- ✓ File Storage → Cloudinary, AWS S3, or similar
- ✓ Domain & SSL → Namecheap, GoDaddy, Cloudflare
- ✓ CI/CD → GitHub Actions, GitLab CI

2 Environment Variables

Never hardcode sensitive information. Use .env files for local development and set environment variables in the platform dashboard for production.

! Never commit .env files to GitHub!

Backend (.env)

```
PORT=5000
MONGO_URI=mongodb+srv://user:pass@cluster.mongodb.net/db
JWT_SECRET=your_secret_key
CLOUDINARY_CLOUD_NAME=xxx
CLOUDINARY_API_KEY=xxx
CLOUDINARY_API_SECRET=xxx
NODE_ENV=production
```

3 Deploy Frontend (Vercel)

- 1 Push your code to GitHub.
- 2 Go to <https://vercel.com> and import your repo.
- 3 Set environment variables (e.g. VITE_API_URL).
- 4 Click Deploy.
- 5 Your app will be live at a .vercel.app domain (or custom domain).

Frontend Environment Variable (.env)

VITE_API_URL=<https://api.yourdomain.com>

4 Deploy Backend (Render)

- 1 Push your code to GitHub.
- 2 Go to <https://render.com> and create a new Web Service.
- 3 Select your repo and set build command and start command.
- 4 Add environment variables (same as .env).
- 5 Deploy and get your backend URL.

Render Settings (Example)

Build Command : `npm install`
Start Command : `npm start`
Environment : Node.js
Instance Type : Free (or Paid)

You can also use Railway, Fly.io or AWS EC2

5 MongoDB Atlas Setup

- 1 Go to <https://www.mongodb.com/atlas>
- 2 Create a free cluster.
- 3 Whitelist your IP (or allow 0.0.0.0/0).
- 4 Create a database user.
- 5 Get the connection string and use it in MONGO_URI.



6 File Uploads (Cloudinary)

- 1 Create an account at <https://cloudinary.com>
- 2 Get Cloud Name, API Key and API Secret.
- 3 Add them to your backend environment variables.
- 4 Use the SDK to upload images in production.



```
import { v2 as cloudinary } from 'cloudinary';
cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET,
});
```

7 Domain & HTTPS

- ✓ Buy a domain from Namecheap / GoDaddy.
- ✓ Point your domain to Vercel (frontend) and Render (backend).
- ✓ Enable HTTPS (automatic in most platforms).
- ✓ Use a custom domain for professional look.



www.yourdomain.com → Frontend (Vercel).

api.yourdomain.com → Backend (Render)

8 CI/CD with GitHub Actions (Optional)

- ✓ Automatically deploy on every push.
- ✓ Example workflow for backend (Render deployment).

```
# .github/workflows/deploy.yml
name: Deploy Backend
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to Render
      - run: curl -X POST ${{ secrets.RENDER_DEPLOY_HOOK }}
```

Drear
Build
Deploy
Repeat
😊

Dream
Build
Deploy
(Netent)
😊